CENTRO UNIVERSITÁRIO FEI

Avenida Humberto de Alencar Castelo Branco, 3972, CEP: 09850-901 São Bernardo do Campo
Telefone: (011) 4353-2900 Fax (011) 4109-5994

Curso de Ciências da Computação

Construção de Serviços

Autores: Pedro Custódio, Kawan Mark, Alexandre Pierri, Lucas Roberto	Data de emissão: 29/04/2026
Revisor:	Data de revisão

Índice

1. OBJETIVO DO DOCUMENTO 3

2. ESTILO DE COORDENAÇÃO DOS SERVIÇOS 3

3. ESTRATÉGIA DE TESTE 4

TIPO DE TESTE: TESTE UNITÁRIO 4
TIPO DE TESTE: TESTE DE INTEGRAÇÃO 4

4. EVIDÊNCIAS DOS TESTES REALIZADOS (PRINTS) 5

1. Objetivo do Documento

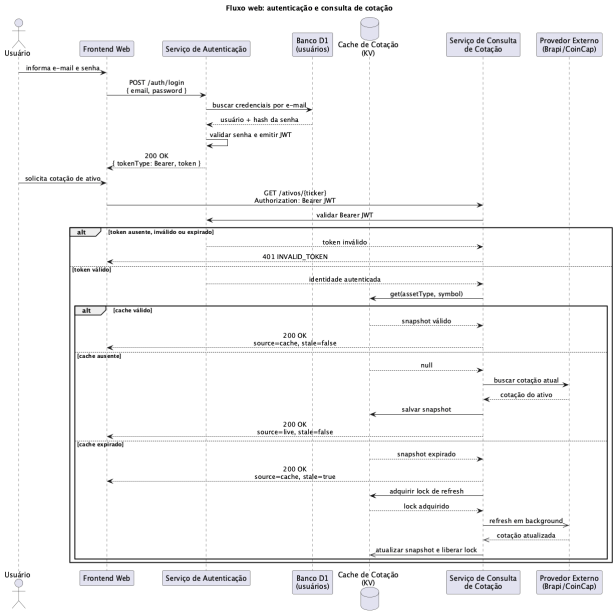
Este documento tem como objetivo descrever a implementação de dois serviços em uma arquitetura orientada a serviços, incluindo a definição do estilo de comunicação entre serviços, justificativa arquitetural e definição de estratégias de teste.

2. Estilo de Coordenação dos Serviços

O estilo de coordenação dos serviços é a orquestração.
O backend controla explicitamente a sequência do fluxo, para consultar uma cotação, o cliente primeiro precisa estar autenticado, o endpoint de cotação valida o token JTW e depois aciona o serviço responsável por consultar preço, cache e provedor externo, portanto a ordem das chamadas é centralizada pela API, e não distribuída em eventos independentes entre serviços.

Fluxo principal:
1. O usuário informa email e senha
2. O frontend envia POST /auth//login ao Serviço de Autenticação.
3. O Serviço de Autenticação consulta o bando de dados, valida a senha e emite um token JWT.
4. O frontend envia GET /ativos/{ticker} com Authorization: Bearer<token?
5. O endpoint de cotação valida o token antes de consultar o ativo.
6. O Serviço de Consulta de Cotação verifica o cache.
7. Se houver cache válido, retorna a cotação em cache.
8. Se o cache estiver ausente, chama o provedor externo, salva o snapshot e retorna a cotação ao cliente.
9. Se o cache estiver expirado, retorna o snapshot stale imediatamente e agenda atualização em background

Diagrama de sequência



3. Estratégia de Teste

Tipo de Teste: Teste Unitário

Serviço de Consulta de Cotação (@finta/price-query)

Objetivo do Teste:	**Validar a lógica interna de consulta de cotação, principalmente a decisão entre buscar dados no cache ou no provedor externo.**
Técnica:	**Testes automatizados com Vitest usando mocks para o gateway de dados de mercado e uma implementação em memória do cache, dessa forma, a regra de negócio é validada sem depender do KV cache da cloudflare ou dos provedores externos reais**
Critério de Finalização	**Todos os testes unitários do package @finta/price-query devem passar (packages/price-query/src/application/price-query-service.test.ts)**

Tipo de Teste: Teste de Integração

Serviço de Autenticação + Serviço de Consulta de Cotação

Objetivo do Teste:	**Validar que a consulta de cotação só é executada quando a autenticação é válida e que o fluxo integrado retorna cotação com metadados de cache.**
Técnica:	**Testes automatizados com Viteste no worker HTTP (apps/backend-cloudflare). O teste aciona enpoints reais da aplicação no ambiente de teste, com versões falsas do banco de dados e cache KV, e provedores externos mockados, isso valida a integração entre autenticação, autorização consulta de**
Critério de Finalização	todos os testes do backend (apps/backend-cloudflare/ src/index.test.ts.) devem passar, incluindo os cenários de login/autenticação, consulta de cotação autenticada, uso de cache e respostas 401 para tokens inválidos.

4. Evidências dos testes realizados (prints)

Teste Unitário

```
> @finta/price-query@0.0.0 test /Users/pedro/Faculdade/topicos-avancados-eng-software/projeto-finta/packages/price-query
> vitest run


 RUN  v3.2.4 /Users/pedro/Faculdade/topicos-avancados-eng-software/projeto-finta/packages/price-query


 ❯ src/application/price-query-service.test.ts [queued]

 Test Files 0 passed (1)
      Tests 0 passed (0)
   Start at 22:11:57
   Duration 104ms
 ✓ src/application/price-query-service.test.ts (6 tests) 4ms
   ✓ PriceQueryService > returns a live quote on cold cache 2ms
   ✓ PriceQueryService > returns a cached quote on warm cache 0ms
   ✓ PriceQueryService > marks cached quotes as stale once ttl expires 0ms
   ✓ PriceQueryService > refreshes stale quotes in the background 0ms
   ✓ PriceQueryService > returns null when cached quote is missing 0ms
   ✓ PriceQueryService > searches cached quotes by asset type and limit 0ms

 Test Files  1 passed (1)
      Tests  6 passed (6)
   Start at  22:11:57
   Duration  223ms (transform 28ms, setup 0ms, collect 27ms, tests 4ms, environment 0ms, prepare 36ms)
```

Teste de integração

```
> backend-cloudflare@ test /Users/pedro/Faculdade/topicos-avancados-eng-software/projeto-finta/apps/backend-cloudflare
> vitest run


 RUN  v3.2.4 /Users/pedro/Faculdade/topicos-avancados-eng-software/projeto-finta/apps/backend-cloudflare


 ❯ src/lib/password.test.ts [queued]

 Test Files 0 passed (7)
      Tests 0 passed (0)
   Start at 22:11:58
   Duration 103ms
 ✓ src/lib/api-key.test.ts (4 tests) 2ms
 ✓ src/lib/jwt.test.ts (4 tests) 4ms
 ✓ src/adapters/price-query/brapi-market-data-gateway.test.ts (5 tests) 5ms
 ✓ src/adapters/price-query/coincap-market-data-gateway.test.ts (6 tests) 5ms
 ✓ src/lib/auth.test.ts (5 tests) 6ms

 ❯ src/index.test.ts [queued]
 ❯ src/lib/password.test.ts 1/2

 Test Files 5 passed (7)
      Tests 25 passed (26)
   Start at 22:11:58
   Duration 202ms
 ✓ src/lib/password.test.ts (2 tests) 50ms

 ❯ src/index.test.ts 0/68

 Test Files 6 passed (7)
      Tests 26 passed (94)
   Start at 22:11:58
   Duration 402ms

 ❯ src/index.test.ts 1/68

 Test Files 6 passed (7)
      Tests 27 passed (94)
   Start at 22:11:58
   Duration 503ms
 ✓ src/index.test.ts (68 tests) 131ms

 Test Files  7 passed (7)
      Tests  94 passed (94)
   Start at  22:11:58
   Duration  572ms (transform 304ms, setup 0ms, collect 547ms, tests 204ms, environment 1ms, prepare 297ms)
```